

THỰC HÀNH TUẦN 3

STREAMS

1. Đọc Chép file txt

```
import java.io.*;
class FileStreamsTest {
    public static void main(String[] args) {
        try {
            File inputFile = new File("in.txt");
            File outputFile = new File("out.txt");

            FileInputStream fis = new FileInputStream(inputFile);
            FileOutputStream fos = new FileOutputStream(outputFile);
            int c;

            while ((c = fis.read()) != -1) {
                fos.write(c);
            }

            fis.close();
            fos.close();
        } catch (FileNotFoundException e) {
            System.err.println("FileStreamsTest: " + e);
        } catch (IOException e) {
            System.err.println("FileStreamsTest: " + e);
        }
    }
}
```

2. Đọc ghi đối tượng tự tạo

- + Sinh viên tìm hiểu giao tiếp Serializable
- + ObjectInputStream/ ObjectOutputStream

```
//----- DOC DOI TUONG // file Student.java-----
import java.io.Serializable;
public class Student implements Serializable {
    private String name;
    private int age;
    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
    public String toString()
    {
        String ret = "My name is " + name + "\nI am " + age + " years old";
        return ret;
    }
}
```

```
//-----/ file WriteMyObject.java
import java.io.*;
public class WriteMyObject {
    public static void main(String[] args) {
        try //ghi du lieu
        {
            FileOutputStream f = new FileOutputStream("student1.dat");
            ObjectOutputStream oStream = new ObjectOutputStream(f);
            Student x = new Student("Bill Gates", 18);
            oStream.writeObject(x);
            oStream.close();
        } catch (IOException e) {
            System.out.println("Error IO file");
        }
        //doc du lieu
        try {
            FileInputStream g = new FileInputStream("student1.dat");
            ObjectInputStream inStream = new ObjectInputStream(g);
            Student y = (Student) inStream.readObject();
            System.out.println(y.toString());
            inStream.close();
        } catch (ClassNotFoundException e) {
            System.out.println("Class not found");
        } catch (IOException e) {
            System.out.println("Error IO file");
        }
    }
}
```

//-----

3. **BufferedReader**

```
import java.io.*;
public class TestInput
{
    public static void main(String[] args) throws Exception
    {
        BufferedReader inStream =
            new BufferedReader(new InputStreamReader(System.in));
        System.out.print("Nhap mot so nguyen:");
        String siNumber = inStream.readLine();
        int iNumber = Integer.parseInt(siNumber);
        System.out.print("Nhap mot so thuc:");
        String sfNumber = inStream.readLine();
    }
}
```

```
float fNumber = Float.parseFloat(sfNumber);
```

```
System.out.println("So nguyen:" + iNumber);
```

```
System.out.println("So thuc:" + fNumber);
```

```
}
```

```
}
```

```
//-----
```

4. PipedOutputStream

```
import java.io.*;
```

```
public class Available {
```

```
    public static void main(String args[]) throws Exception {
```

```
        byte[] b = {1, 4, 67, 4, 6};
```

```
        PipedOutputStream poStream = new PipedOutputStream();
```

```
        PipedInputStream piStream = new PipedInputStream();
```

```
        //piped input stream connect to the piped output stream
```

```
        piStream.connect(poStream);
```

```
        //Writes specified byte array.
```

```
        poStream.write(b, 1, 2);
```

```
        //number of bytes that can be read from this input stream.
```

```
        System.out.println("No. of byte Available " + piStream.available());
```

```
        // Closes piped input stream
```

```
        poStream.close();
```

```
        // Closes piped output stream
```

```
        piStream.close();
```

```
}
```

```
//-----
```

5. ByteArrayInputStream

```
import java.io.*;
```

```
class ByteArrayInputStreamReset {
```

```
    public static void main(String args[]) throws IOException {
```

```
        String tmp = "aBC";
```

```
        byte b[] = tmp.getBytes();
```

```
        ByteArrayInputStream in = new ByteArrayInputStream(b);
```

```
        for (int i=0; i<2; i++) {
```

```
            int c;
```

```

while ((c = in.read()) != -1) {
    if (i == 0) {
        System.out.print((char) c);
    } else {
        System.out.print(Character.toUpperCase((char) c));
    }
}
System.out.println();
in.reset();
}
}
}
//-----

```

6. DataInputStream

```

import java.io.DataInputStream;
import java.io.FileInputStream;
import java.io.DataOutputStream;
import java.io.FileOutputStream;
import java.io.IOException;

public class PrimitiveStreamExample {

    public static void main(String[] args) {
        //
        // Prepares some data to be written to a file.
        //
        int cityIdA = 1;
        String cityNameA = "Gree Th?n T?c";
        int cityPopulationA = 500000;
        float cityTempA = 15.50f;

        int cityIdB = 2;
        String cityNameB = "Salt Lake City";
        int cityPopulationB = 250000;
        float cityTempB = 10.45f;

        try {
            //
            // Create an instance of FileOutputStream with cities.dat
            // as the file name to be created. Then we pass the input
            // stream object in the DataOutputStream constructor.
            //
            FileOutputStream fos = new FileOutputStream("cities.txt");
            DataOutputStream dos = new DataOutputStream(fos);

            //
            // Below we write some data to the cities.dat.
            // DataOutputStream class have various method that allow

```

```

// us to write primitive type data and string. There are
// method called writeInt(), writeFloat(), writeUTF(),
// etc.
//
dos.writeInt(cityIdA);
dos.writeUTF(cityNameA);
dos.writeInt(cityPopulationA);
dos.writeFloat(cityTempA);

dos.writeInt(cityIdB);
dos.writeUTF(cityNameB);
dos.writeInt(cityPopulationB);
dos.writeFloat(cityTempB);

dos.flush();
dos.close();

//
// Now we have a cities.dat file with some data in it.
// Next you'll see how easily we can read back this
// data and display it. Just like the DataOutputStream
// the DataInputStream class have the corresponding
// read methods to read data from the file. Some of
// the method names are readInt(), readFloat(),
// readUTF(), etc.
//
FileInputStream fis = new FileInputStream("cities.txt");
DataInputStream dis = new DataInputStream(fis);

//
// Read the first data
//
int cityId1 = dis.readInt();
System.out.println("Id: " + cityId1);
String cityName1 = dis.readUTF();
System.out.println("Name: " + cityName1);
int cityPopulation1 = dis.readInt();
System.out.println("Population: " + cityPopulation1);
float cityTemperature1 = dis.readFloat();
System.out.println("Temperature: " + cityTemperature1);

//
// Read the second data
//
int cityId2 = dis.readInt();
System.out.println("Id: " + cityId2);
String cityName2 = dis.readUTF();
System.out.println("Name: " + cityName2);
int cityPopulation2 = dis.readInt();

```

```

        System.out.println("Population: " + cityPopulation2);
        float cityTemperature2 = dis.readFloat();
        System.out.println("Temperature: " + cityTemperature2);
    } catch (IOException e) {
        e.printStackTrace();
    }
}
}
//-----

```

7. StringArrayInputStream

```

import java.io.*;

public class StringArrayInputStreamReset {

    public static void main(String[] args) throws Exception{

        // Create object of StringBufferInputStream class
        StringBufferInputStream obj = new StringBufferInputStream("Mahendra");
        // read() method call. This is deprecated method in java 1.6 API.
        System.out.println("First character of the buffer : " +(char)obj.read());
        System.out.println("Next character of the buffer : " +(char)obj.read());
        System.out.println("Next character of the buffer : " +(char)obj.read());

        // reset() method call. This is deprecated method in java 1.6 API.
        obj.reset();
        System.out.println("After reset the buffer,next character is : " +(char)obj.read());
    }
}
//-----

```

Bài tập

- Viết chương trình mycopy sử dụng như sau: java mycopy filename1 filename2
 - Nếu filename1 và filename2 là 2 file thì chương trình copy nội dung của filename1 sang filename2
 - Nếu filename2 là thư mục thì copy filename1 sang thư mục filename2
 - Nếu filename1 có tên là con thì cho phép tạo filename2 với nội dung gõ từ bàn phím (giống lệnh copy con).
- Viết chương trình hiển thị nội dung của file văn bản trong đối tượng TextArea.